

TETI LAB SKILLS - PEMROGRAMAN DASAR
Hassan Hamka Ahmad - Teknologi Informasi - 26/588728/TK/67187

Problem 1

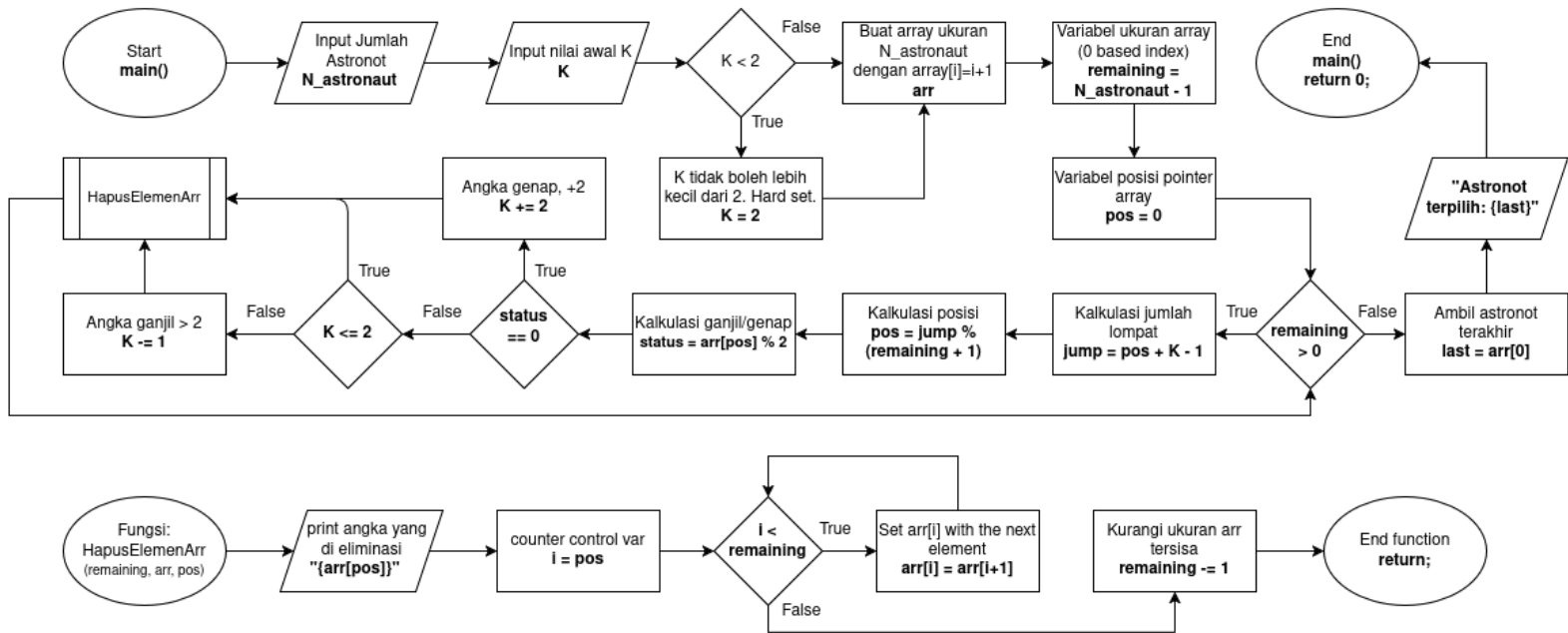


Fig. 1. Flowchart workflow of the C++ code solving Problem 1: The Last Astronaut

Problem 1 menyatakan sebuah masalah dimana terdapat sebuah array berukuran N yang mana elemennya akan diproses satu-persatu dengan nilai K hingga satu elemen tersisa. Sistem eliminasi dengan nilai K bermula dari posisi index pertama #0 melompat sejauh K dimana K=1 adalah index yang sama. Maka K = 2 adalah elemen setelah elemen posisi saat ini. K tidak boleh kurang dari 2 sesuai dengan aturan masalah.

Solusi yang saya rancang bermula dengan mengambil nilai awal K dan N, kemudian membuat sebuah array dengan N-1 jumlah elemen dimana $arr[i]$ bernilai $i+1$. Walau ini dapat dianggap tidak efisien, menurut saya ini adalah satu-satunya cara mudah untuk menyelesaikan masalah ini.

Proses logika dimulai dengan memasuki sebuah loop hingga jumlah elemen tersisa adalah tepat 1. Melompat dari posisi #0 menuju #(posisi saat ini + K - 1), posisi sekarang adalah posisi angka yang akan dieliminasi. Mengecek angka yang akan dieliminasi, jika angka ini bersifat ganjil, maka K - 1 kecuali jika K = 2 agar tidak kurang dari 2. Jika angka ini genap, maka K + 2. Kemudian proses menghapus elemen ini dijalankan melalui subprogram (fungsi) terpisah.

Karena kita hanya dapat menggunakan library iostream, kita perlu membuat sebuah algoritma “penghapus” elemen dalam array. Dengan membuat elemen posisi saat ini $arr[i] =$

arr[i+1] hingga ukuran array saat ini, kita dapat secara teknis memundurkan semua elemen di depan posisi saat ini hingga jumlah ukuran array saat ini. Setelah subproses ini selesai, ukuran array dikurangi 1.

Semua logika proses di proses utama diulang hingga elemen tersisa hanyalah 1. Yang mana elemen tersebut adalah elemen terpilih menurut peraturan problem 1.

Problem 2

DEKLARASI

```
sLen      : integer
str       : string
prevVal   : integer
i         : integer
nilaiHuruf : integer
```

DESKRIPSI

```
Tulis "Panjang string: "
Baca sLen
```

```
Jika sLen <= 0 maka
    Tulis "Panjang string tidak boleh 0 atau kurang"
    Selesai
```

```
Tulis "Masukkan string dalam huruf kapital: "
Baca str
```

```
// Huruf pertama tidak mengalami perubahan
prevVal <- kode ASCII(str[0]) - kode ASCII('A') + 1
```

```
Untuk i <- 1 sampai sLen - 1 lakukan
    nilaiHuruf <- kode ASCII(str[i]) - kode ASCII('A') + 1
```

```
    // Geser huruf berdasarkan nilai huruf sebelumnya
    nilaiHuruf <- nilaiHuruf + prevVal
```

```
    Selama nilaiHuruf > 26 lakukan
        nilaiHuruf <- nilaiHuruf - 26
```

```
    str[i] <- karakter ASCII('A') + nilaiHuruf - 1
```

```
    // Simpan nilai huruf yang telah diubah
    prevVal <- nilaiHuruf
```

```
Tulis "String hasil: ", str
```

SELESAI

Fig. 2. Pseudocode of the C++ code solving Problem 2: Alien-In-The-Middle

Problem 2 menyatakan sebuah masalah dimana terdapat sebuah string dengan panjang tertentu yang seluruh elemennya berupa huruf kapital. Setiap huruf pada string akan diproses berdasarkan nilai alfabet dari huruf sebelumnya, dengan A = 1, B = 2, hingga Z = 26. Huruf pertama tidak mengalami perubahan, sedangkan setiap huruf berikutnya digeser maju sebanyak nilai huruf tepat sebelumnya. Jika hasil pergeseran melewati Z, perhitungan kembali dimulai dari A.

Solusi yang saya rancang bermula dengan mengambil nilai panjang string dan string yang akan diproses. Jika panjang string kurang dari atau sama dengan 0, program akan menghentikan proses karena panjang string tidak valid. Setelah itu, string disimpan dalam array karakter dinamis. Nilai huruf pertama dihitung menggunakan kode ASCII, yaitu dengan mengurangi kode ASCII huruf tersebut dengan 64 sehingga A bernilai 1 dan Z bernilai 26.

Proses logika dimulai dengan memproses setiap huruf setelah huruf pertama menggunakan sebuah loop. Nilai huruf sebelumnya ditambahkan ke kode ASCII huruf saat ini. Jika hasil penjumlahan melewati kode ASCII Z, nilai tersebut dikurangi 26 hingga kembali berada dalam rentang huruf kapital A sampai Z. Setelah huruf berhasil diubah, nilai huruf hasil perubahan tersebut disimpan sebagai nilai untuk proses huruf berikutnya. Proses ini diulang hingga seluruh string selesai diproses, kemudian string hasil sandi ditampilkan dan memori array dibebaskan.